

Place & Route: Del Netlist al Layout

Diseño e Implementación de Circuitos Digitales en VLSI

Agustín Pesce — Becas de Grado 2026

2026

Contents

1	Contexto: el lugar de P&R en el flujo	3
1.1	El flujo completo de diseño ASIC	3
1.2	¿Qué entra al P&R?	3
1.3	¿Qué sale del P&R?	3
1.4	Síntesis vs P&R	4
2	Floorplanning	4
2.1	¿Qué es el floorplan?	4
2.2	Utilización del core	4
2.3	Core offset	5
2.4	Comando en Fusion Compiler	5
2.5	Power Planning	5
2.6	Pin Placement	5
3	Placement	6
3.1	¿Qué es placement?	6
3.2	Site Rows	6
3.3	Timing-Driven Placement	7
3.4	Congestion-Driven Placement	7
3.5	Legalización	7
3.6	place_opt en Fusion Compiler	7
4	Clock Tree Synthesis (CTS)	8
4.1	El problema del clock skew	8
4.2	El árbol de clock	8
4.3	Parámetros del CTS	9
4.4	clock_opt en Fusion Compiler	9
4.5	H-Tree: topología balanceada	9
5	Routing	10
5.1	¿Qué es routing?	10
5.2	Capas de metal en SAED 32nm	10
5.3	Global Routing vs Detailed Routing	10
5.4	Design Rules (DRC)	11
5.5	Congestion	11
5.6	Antenna Effect	12
5.7	route_opt	12

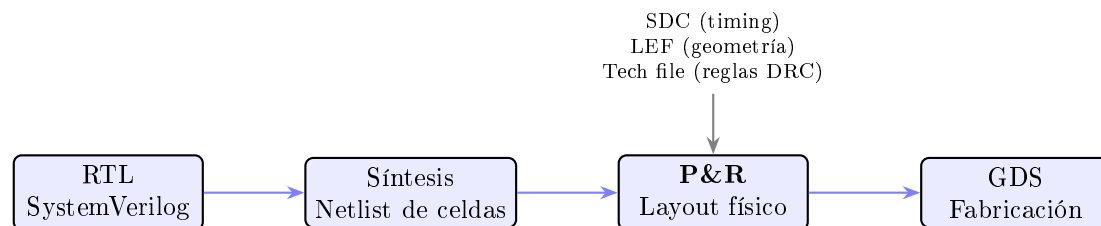
6 Signoff	12
6.1 STA post-routing	12
6.2 IR Drop	12
6.3 Fill Cells	13
6.4 GDS: el archivo final	13
6.5 LVS: Layout vs Schematic	13
6.6 El flujo completo en pnr.tcl	14
7 Resultados reales: el contador de 8 bits	14
7.1 Floorplan	14
7.2 Placement	15
7.3 Clock Tree	15
7.4 Routing	15
7.5 QoR post-routing	15
8 Diferencias entre diseños: contador vs filtro FIR	15
9 Ejercicios	16
9.1 Ejercicio 1: Dimensionamiento del floorplan	16
9.2 Ejercicio 2: Análisis de placement	16
9.3 Ejercicio 3: Clock Tree	17
9.4 Ejercicio 4: Routing y DRC	18
9.5 Ejercicio 5: Comparación pre/post routing	18
9.6 Ejercicio Integrador: Filtro FIR de 19 taps — Flujo completo	19
10 Resumen del flujo P&R	20

1 Contexto: el lugar de P&R en el flujo

1.1 El flujo completo de diseño ASIC

Hasta ahora trabajamos con síntesis: transformamos RTL (SystemVerilog) en un netlist de celdas estándar. El sintetizador eligió qué compuertas y flip-flops usar, pero no dijo **dónde** van físicamente en el chip ni **cómo** se conectan con wires de metal.

Place & Route (P&R) es el paso que transforma ese netlist abstracto en un **layout físico real**: coordenadas exactas de cada celda, capas de metal, vias. El resultado final es un archivo GDS que va a la fábrica.



1.2 ¿Qué entra al P&R?

1. **Netlist de síntesis** (.v o .nlib): las celdas estándar y sus conexiones lógicas.
2. **SDC**: las constraints de timing (clock, input/output delays). El mismo archivo que usó síntesis.
3. **LEF** (Library Exchange Format): describe la geometría física de cada celda (tamaño, posición de pines, capas de metal).
4. **Tech file**: las reglas de fabricación (anchos mínimos de metal, espaciado mínimo, capas disponibles).
5. **TLU+** (Parasitic models): modelos de resistencia e inductancia de los wires de cada capa de metal.

1.3 ¿Qué sale del P&R?

1. **GDS** (Graphic Design System): el layout completo en formato estándar para la foundry.
2. **Netlist post-routing**: el netlist con buffers y celdas adicionales insertadas por el P&R.
3. **SDF** (Standard Delay Format): delays reales de cada wire, para simulación post-layout.
4. **DEF** (Design Exchange Format): posiciones de todas las celdas.
5. **SPEF** (Standard Parasitic Exchange Format): parásitos reales para STA post-routing.

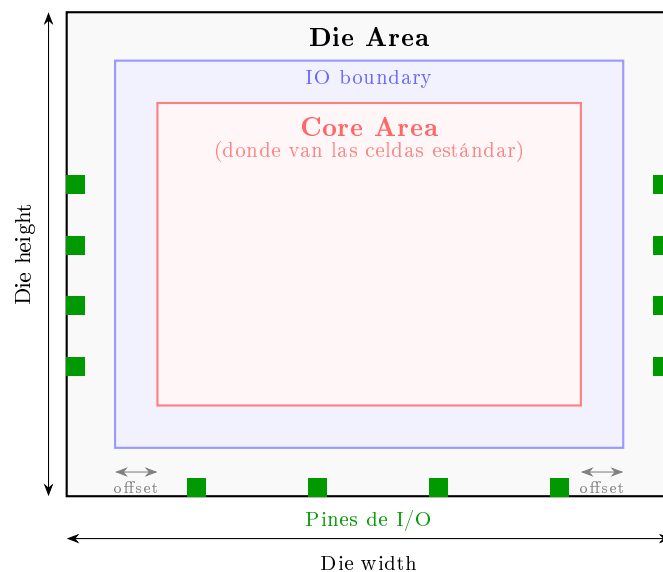
1.4 Síntesis vs P&R

	Síntesis	P&R
Mundo	Lógico/abstracto	Físico/real
Coordenadas	No (celdas sin posición)	Sí (cada celda tiene X,Y)
Wires	Ideales (sin delay)	Reales (RC parásitos)
Timing	Estimado	Exacto
DRC	No aplica	Obligatorio
Entrada	RTL	Netlist
Salida	Netlist	GDS
Tool	Fusion Compiler (syn)	Fusion Compiler (pnr) / ICC2

2 Floorplanning

2.1 ¿Qué es el floorplan?

El floorplan define la **geometría del chip**: el tamaño total (die area), el área donde van las celdas estándar (core area), los márgenes para los pines de I/O, y la distribución de bloques si hay jerarquía.



2.2 Utilización del core

La **utilización** es el porcentaje del core area que ocupan las celdas estándar:

$$\text{Utilización} = \frac{\text{Área total de celdas}}{\text{Core Area}} \times 100\%$$

No podés llegar a 100% — necesitás espacio para los wires. En la práctica:

Utilización	Consecuencia
< 40%	Demasiado espacio, chip muy grande, desperdicio de área
50% – 70%	Rango típico para diseños normales
70% – 80%	Congestionado, el router puede tener problemas
> 85%	Muy difícil de rutear, alto riesgo de DRC violations

En nuestro `pnr.tcl` usamos `core_utilization 0.5` (50%). Esto significa que el core tiene el doble del área mínima necesaria para las celdas.

2.3 Core offset

El `core_offset {2 2 2 2}` especifica el margen entre el borde del die y el borde del core, en micrómetros, en el orden: {left bottom right top}. Este espacio se usa para los pines de I/O y las stripes de power.

2.4 Comando en Fusion Compiler

```
# Forma basica: solo especificar utilizacion y offset
initialize_floorplan -core_utilization 0.5 -core_offset {2 2 2 2}

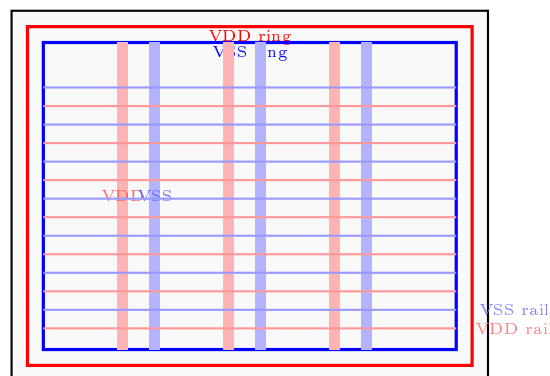
# Forma con dimensiones fijas (en micrones)
initialize_floorplan -core_size {50 50} -core_offset {2 2 2 2}

# Ver el resultado
report_floorplan
```

2.5 Power Planning

La red de alimentación (power network) distribuye VDD y VSS a todas las celdas. Un diseño real tiene:

- **Power rings:** anillos de metal alrededor del core para VDD y VSS.
- **Power stripes:** líneas horizontales y verticales de metal grueso que recorren el core.
- **Power rails:** líneas de M1 en cada fila de celdas (VDD arriba, VSS abajo de cada fila).



En el flujo del curso, el power planning es simplificado:

```
# Conectar pines VDD y VSS de todas las celdas a las redes de power
connect_pg_net -net VDD [get_pins -hierarchical */VDD]
connect_pg_net -net VSS [get_pins -hierarchical */VSS]
```

2.6 Pin Placement

Los pines de I/O son los puntos de conexión del chip con el exterior. Hay que especificar en qué capas de metal y en qué lados del chip van:

```
# Todos los pines en capas M3 o M4
set_block_pin_constraints -allowed_layers {M3 M4} -self

# Pines de entrada en el lado izquierdo (side 1), solo en M3
set_individual_pin_constraints -ports {i_clk i_rst i_enable} \
    -allowed_layers {M3} -sides 1

# Pines de salida en el lado derecho (side 3), solo en M3
set_individual_pin_constraints -ports {o_count[0] o_count[1] ...} \
    -allowed_layers {M3} -sides 3

# Colocar los pines automaticamente
place_pins -self
```

Los lados son: 1=izquierdo, 2=abajo, 3=derecho, 4=arriba.

3 Placement

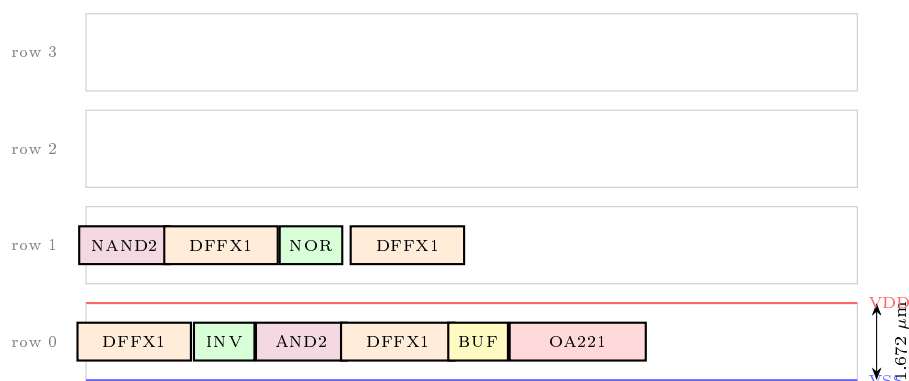
3.1 ¿Qué es placement?

Placement es el proceso de asignar una **posición física** (x, y) a cada celda estándar dentro del core. El placer tiene que satisfacer varias condiciones simultáneamente:

1. Las celdas tienen que estar dentro del core area.
2. Las celdas tienen que alinearse con las **site rows** (filas de altura fija).
3. No puede haber superposición entre celdas.
4. El timing tiene que cumplirse (celdas del critical path cerca una de la otra).
5. La congestión tiene que ser controlable (no demasiados wires en una zona).

3.2 Site Rows

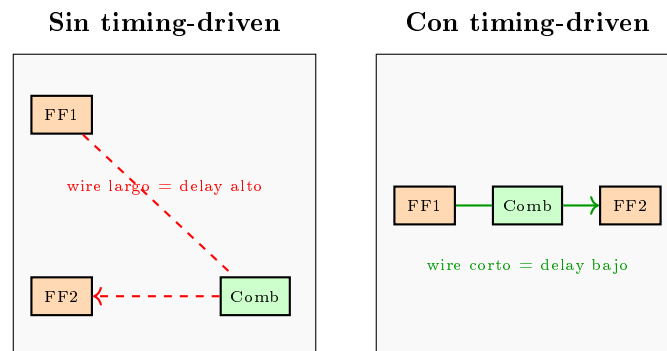
Las celdas estándar tienen **altura fija** — en SAED 32nm es $1.672 \mu\text{m}$. El core está dividido en filas de esa altura. Cada celda ocupa una o más “sites” (unidades de ancho) dentro de una fila.



Las celdas adyacentes comparten los rails de VDD y VSS — por eso las filas alternas se pueden voltear (flip) verticalmente: VDD-VSS-VDD-VSS...

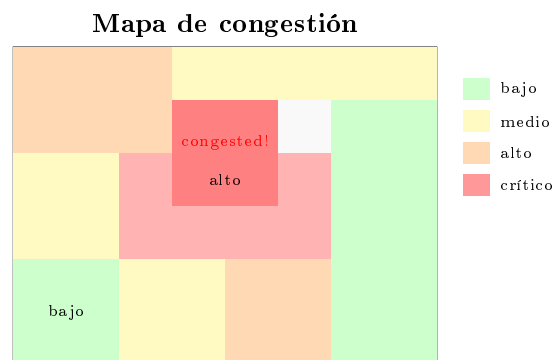
3.3 Timing-Driven Placement

El placer analiza el timing y coloca las celdas del **critical path** cerca entre sí para minimizar el delay de los wires.



3.4 Congestion-Driven Placement

El placer también estima cuántos wires van a pasar por cada zona del chip. Si una zona tiene demasiada demanda de routing, distribuye las celdas para evitar el congestionamiento.



3.5 Legalización

Después del placement inicial, puede haber celdas que se superponen o que no están alineadas con las site rows. La **legalización** mueve cada celda a la posición legal más cercana sin violar las reglas.

El report de legalización muestra el desplazamiento (displacement) de cada celda:

```
number of cells:      31
max cell displacement: 0.88 um (0.53 row heights)
avg cell displacement: 0.40 um (0.24 row heights)
number of cells moved: 23
```

Si los desplazamientos son grandes, hay un problema con el floorplan (utilización demasiado alta, o floorplan muy pequeño).

3.6 place_opt en Fusion Compiler

place_opt es el comando principal que hace placement + legalización + optimización de timing juntos:

```
# Deshabilitar CCD para evitar conflictos con clock_opt
set_app_option -name place_opt.flow.enable_ccd -value false

# Forzar placement de celdas que quedaron sin ubicar
foreach_in_collection cell [get_cells -filter "is_placed == false"] {
    set_attribute $cell physical_status placed
}

# Correr placement + optimizacion
place_opt
```

Después de place_opt podés ver el QoR:

WNS = 0.000 TNS = 0.000 Area = 106.49 InstCnt = 31

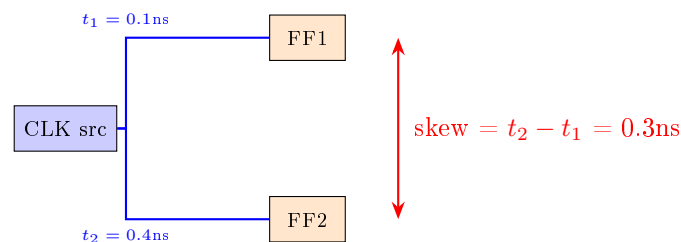
WNS (Worst Negative Slack) = 0 significa que no hay violaciones de timing post-placement.

4 Clock Tree Synthesis (CTS)

4.1 El problema del clock skew

En un diseño síncrono, el clock tiene que llegar a **todos los flip-flops** al mismo tiempo. En la realidad, el clock sale de un solo punto (la fuente) y tiene que llegar a potencialmente miles de flip-flops distribuidos en todo el chip.

Si el clock tarda diferente en llegar a dos flip-flops, hay **skew**:



El skew afecta el timing de dos maneras:

Setup: si el clock llega tarde al flip-flop receptor (FF2 recibe el clock después que FF1), el dato tiene más tiempo para llegar. El skew *relaja* el setup check.

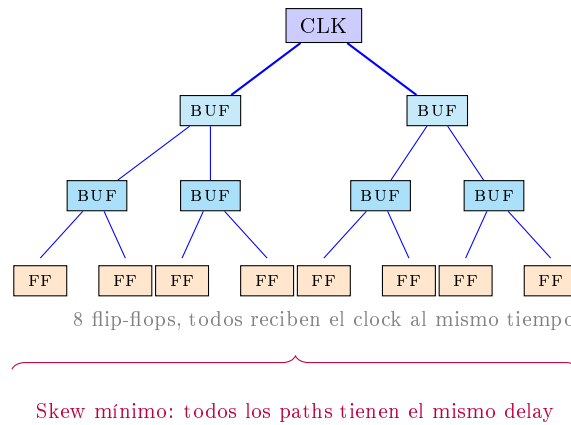
Hold: el mismo skew que relaja setup *empeora* el hold check. Si el clock llega muy tarde a FF2, el dato nuevo puede llegar demasiado rápido y pisarle al viejo.

$$\text{Setup slack} = T_{\text{period}} + \text{skew}_{\text{til}} - T_{\text{clk} \rightarrow Q} - T_{\text{comb}} - T_{\text{setup}}$$

$$\text{Hold slack} = T_{\text{clk} \rightarrow Q} + T_{\text{comb}} - T_{\text{hold}} - \text{skew}_{\text{daino}}$$

4.2 El árbol de clock

La solución es construir un **árbol de buffers** que reparta el clock de forma balanceada. Cada nivel del árbol tiene buffers o inversores que amplifican la señal y la distribuyen:



4.3 Parámetros del CTS

Parámetro	Descripción
Skew	Diferencia de latencia entre los flip-flops más rápido y más lento. Meta: < 50–100ps.
Latencia	Tiempo desde la fuente del clock hasta un flip-flop.
Transition time	Tiempo de subida/bajada de la señal de clock en cada punto.
Fanout	Número de cargas que maneja cada buffer.

4.4 clock_opt en Fusion Compiler

```
# Correr CTS + optimizacion post-CTS
clock_opt

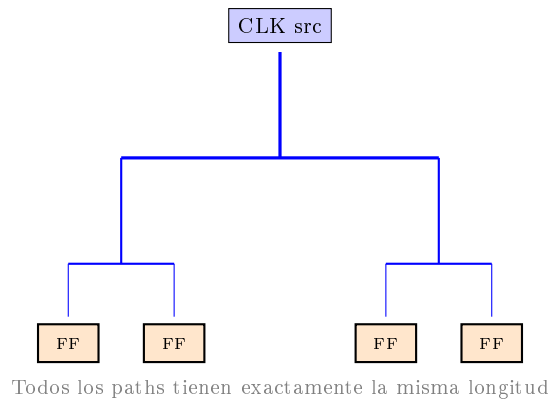
# Reportar el resultado del clock tree
report_clock_timing -type summary
report_clock_timing -type skew
```

Después del CTS el timing cambia porque:

1. Se insertan buffers de clock (agregan latencia y área).
2. El skew real entre flip-flops es diferente al skew ideal (0) que asumía placement.
3. El hold timing puede empeorar si hay mucho skew.

4.5 H-Tree: topología balanceada

La topología más usada es el **H-Tree**: los buffers se organizan en una estructura en H que garantiza que todos los paths tienen la misma longitud de wire.



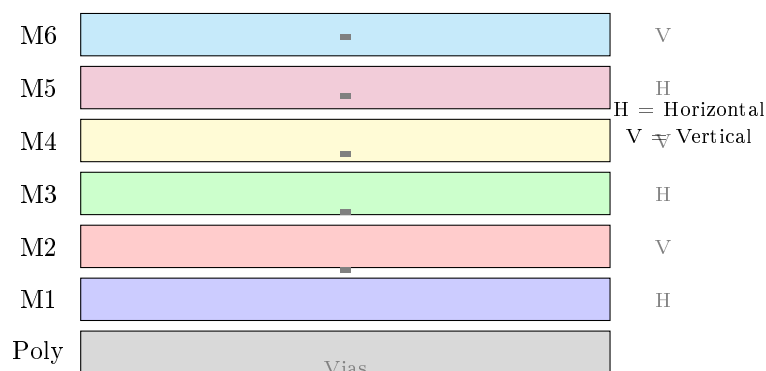
5 Routing

5.1 ¿Qué es routing?

Routing es el proceso de trazar los **wires de metal** que conectan los pines de todas las celdas según la netlist. Cada conexión lógica (net) tiene que convertirse en un camino físico de metal.

5.2 Capas de metal en SAED 32nm

SAED 32nm tiene 9 capas de metal más una capa RDL (redistribution layer):



- **M1**: capa más baja, usada principalmente para conexiones dentro de las celdas.
- **M2**: primera capa de routing libre, dirección vertical.
- **M3**: dirección horizontal.
- **M4–M9**: capas más altas para wires largos (clock, power buses).
- **MRDL**: capa gruesa para bonding wires en el packaging.

Las capas alternan horizontal/vertical para evitar que dos wires de la misma dirección se crucen.

5.3 Global Routing vs Detailed Routing

El routing se hace en dos fases:

Global Routing: planifica las rutas a grandes rasgos, dividiendo el chip en “gcells” (celdas de routing) y asignando cada net a un camino aproximado. No pone wires reales, solo verifica que el routing sea factible y estima la congestión.

Detailed Routing: toma el plan del global routing y pone los wires reales con coordenadas exactas, respetando todas las reglas de diseño (DRC).

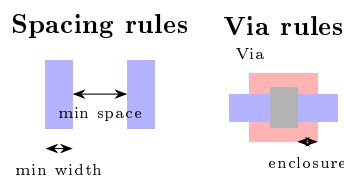
```
# Global + Detailed routing en un paso
route_auto

# Optimizacion post-routing
route_opt
```

5.4 Design Rules (DRC)

Las **design rules** son restricciones físicas impuestas por el proceso de fabricación. Las más importantes:

Regla	Descripción
Min Width	El wire debe tener un ancho mínimo (ej: 56nm en M2 de SAED 32nm).
Min Spacing	Dos wires de la misma capa deben estar separados al menos X nm.
Min Area	El área total de un wire debe ser mínima (evita "dots" de metal).
Via enclosure	El wire debe "encerrar" la via en cada capa.
Antenna Rule	Un wire no puede acumular demasiada carga durante fabricación.



Una **DRC violation** ocurre cuando alguna de estas reglas se viola. El chip con DRC violations no puede fabricarse.

5.5 Congestion

La congestión ocurre cuando una zona del chip tiene más demanda de routing de lo que las capas de metal disponibles pueden satisfacer. El router mide la congestión como:

$$\text{Congestión} = \frac{\text{Demanda de tracks}}{\text{Capacidad de tracks}} \times 100\%$$

Si la congestión supera 100%, el router no puede rutear todos los wires y quedan “open nets” (conexiones sin rutear).

En el flujo del curso, el reporte de congestión post-routing muestra:

```
Average vertical track utilization = 6.32%
Peak vertical track utilization = 47.37%
Average horizontal track utilization = 6.70%
Peak horizontal track utilization = 62.50%
```

Para el contador de 8 bits la congestión es muy baja (menos del 65%). Para diseños más grandes o con utilización alta, puede ser crítico.

5.6 Antenna Effect

Durante la fabricación, los wires de metal actúan como antenas y acumulan cargas. Si el wire es muy largo, la carga acumulada puede dañar los transistores conectados.

La solución es insertar **diodos de antenna** que drenan la carga, o cortar el wire con vias (“metal jumping”).

FC verifica las reglas de antena automáticamente durante el routing.

5.7 route_opt

Después de `route_auto`, `route_opt` hace una segunda pasada optimizando:

- Reemplaza celdas en el critical path por versiones más rápidas.
- Inserta buffers para mejorar el timing.
- Rerutea wires con mayor congestion o DRC.
- Ajusta el clock tree para mejorar hold timing.

6 Signoff

6.1 STA post-routing

Después del routing, el timing se recalcula con los **parásitos reales** de los wires. Los wires tienen resistencia (R) y capacitancia (C) que agregan delay. Este delay no existía en síntesis (donde los wires eran ideales).

El delay de un wire de longitud L en la capa M es aproximadamente:

$$t_{wire} \approx 0.38 \cdot R \cdot C = 0.38 \cdot (r \cdot L) \cdot (c \cdot L) = 0.38 \cdot r \cdot c \cdot L^2$$

Donde r y c son resistencia y capacitancia por unidad de longitud. El delay crece con el **cuadrado** de la longitud del wire.

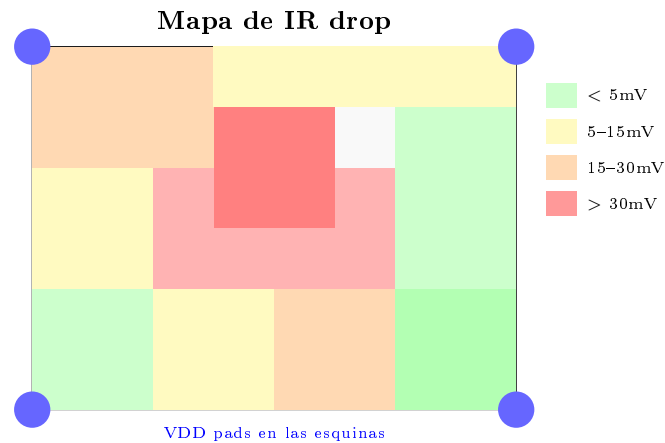
Timing cierra en síntesis pero viola post-routing

Es común que el timing cierre en síntesis (con wires ideales) pero viole después del routing (con parásitos reales). La diferencia se llama “timing degradation”. FC hace estimaciones de parásitos durante el routing para minimizar este problema, pero no siempre alcanza. La solución es hacer “ECO” (Engineering Change Order): optimizaciones post-routing para cerrar timing.

6.2 IR Drop

La red de power tiene resistencia. Cuando las celdas consumen corriente, hay una caída de tensión ($V = I \cdot R$) entre el pin de alimentación y las celdas más alejadas.

Si el IR drop es demasiado grande, las celdas en esas zonas ven una tensión de alimentación menor y se vuelven más lentas. Esto puede causar violaciones de timing que no se ven en el análisis estático.



6.3 Fill Cells

Las filas de celdas estándar tienen que estar completamente llenas — las zonas vacías violan las reglas de densidad de metal. Se insertan **fill cells** (celdas sin función lógica) para llenar los espacios:

```
# Insertar fill cells al final del routing
add_fillers -cell {FILLCELL_X8 FILLCELL_X4 FILLCELL_X2 FILLCELL_X1}
```

6.4 GDS: el archivo final

El GDS (Graphic Design System II) es el formato estándar de la industria para describir el layout de un chip. Contiene:

- Las geometrías de cada capa de metal (polígonos en coordenadas nanométricas)
- Texto (nombres de celdas, pines)
- Referencias a celdas de la librería

El GDS va a la foundry, que lo usa para fabricar las máscaras de litografía y luego el chip.

```
# Exportar GDS
write_gds ${PROJ_ROOT}/pnr/outputs/${TOP}.gds
```

6.5 LVS: Layout vs Schematic

LVS es una verificación que compara el layout (GDS) con el netlist lógico para asegurarse de que sean equivalentes. Verifica:

- Que todos los transistores del netlist están en el layout.
- Que todas las conexiones del netlist están implementadas en el layout.
- Que no hay conexiones extra en el layout que no estén en el netlist (shorts).

Un chip no puede ir a fabricación sin pasar LVS clean.

6.6 El flujo completo en pnr.tcl

```

set TOP contador
set LIB_BASE $::env(LIB_BASE)
set PROJ_ROOT [file normalize [file dirname [info script]]/../../]

# 1. Abrir el diseño post-síntesis
open_lib    ${PROJ_ROOT}/syn/scripts/${TOP}.nlib
open_block ${TOP}

# 2. Floorplan
source ${PROJ_ROOT}/pnr/inputs/${TOP}_floorplan.tcl

# 3. Power connections
connect_pg_net -net VDD [get_pins -hierarchical */VDD]
connect_pg_net -net VSS [get_pins -hierarchical */VSS]

# 4. Placement
foreach_in_collection cell [get_cells -filter "is_placed == false"] {
    set_attribute $cell physical_status placed
}
set_app_option -name place_opt.flow.enable_ccd -value false
place_opt

# 5. Clock Tree Synthesis
set cts_result [catch {clock_opt} cts_err]
if {$cts_result != 0} {
    puts "WARNING: clock_opt returned error: $cts_err"
}

# 6. Routing
route_auto
route_opt

# 7. Reports
report_timing > ${PROJ_ROOT}/pnr/outputs/timing.rpt
report_area > ${PROJ_ROOT}/pnr/outputs/area.rpt
report_power > ${PROJ_ROOT}/pnr/outputs/power.rpt
report_qor > ${PROJ_ROOT}/pnr/outputs/qor.rpt

# 8. Outputs
write_verilog ${PROJ_ROOT}/pnr/outputs/${TOP}_pnr_netlist.v
write_sdc -output ${PROJ_ROOT}/pnr/outputs/${TOP}_pnr.sdc
catch {write_gds ${PROJ_ROOT}/pnr/outputs/${TOP}.gds}
save_lib
echo "=== P&R complete ==="
exit

```

7 Resultados reales: el contador de 8 bits

7.1 Floorplan

Core utilization ratio = 55.34%
 Core area = 195.18 μm^2
 Number of site rows = 8
 Number of pins created = 11

El contador de 8 bits tiene 31 celdas estándar (incluyendo buffers de clock). El core queda a 55% de utilización con el floorplan de `core_utilization 0.5`.

7.2 Placement

```
Place-opt final QoR Summary:
WNS = 0.000    TNS = 0.000    NSV = 0
Area = 106.49   InstCnt = 31    LVT = 0%
```

Post-placement el timing cierra limpio ($WNS = 0$). Todas las celdas son HVT porque el período es de 2ns — hay tiempo de sobra.

7.3 Clock Tree

El contador tiene un solo clock y 8 flip-flops. El CTS es trivial — FC inserta algunos buffers de clock pero el árbol tiene muy poco fanout.

7.4 Routing

```
Average vertical track utilization = 6.32%
Peak    vertical track utilization = 47.37%
Total wire length = 337.52 um
Via VIA12SQ_C count = 76
Via VIA23SQ_C count = 64
```

La utilización de tracks es muy baja — el diseño es pequeño y el router no tiene dificultad. Los wires se distribuyen principalmente en M2 (vertical) y M3 (horizontal).

7.5 QoR post-routing

```
Scenario func_slow:
WNS = +0.01 ns (MET)
TNS = 0.00 ns
Area = 106.49 um^2
Leakage = 9.54 uW
```

El timing cierra post-routing con solo 10ps de slack. El chip es funcional.

8 Diferencias entre diseños: contador vs filtro FIR

Para entender el impacto del P&R en diseños más complejos, comparemos el contador de 8 bits con un filtro FIR de 19 taps:

Métrica	Contador 8b	FIR 19 taps (estimado)
Flip-flops	8	~180 (19 registros $\times$ 8 bits + pipeline)
Celdas totales	31	~800–1200
Área	106 μm^2	~3000–6000 μm^2
Fanout clock	8	~180
Niveles de CTS	1–2	4–6
Skew esperado	<10ps	~30–80ps
Tiempo de P&R	<1 min	~5–15 min

El filtro FIR va a tener:

- Un clock tree más complejo (más fanout, más niveles de buffers).
- Mayor congestión en la zona del sumador acumulador.
- Más sensibilidad a IR drop si la potencia es alta.
- Critical path a través del sumador parcial (como vimos en el barrido de síntesis).

9 Ejercicios

9.1 Ejercicio 1: Dimensionamiento del floorplan

El sintetizador reportó que el filtro FIR de 19 taps tiene un área de celda de **3200 μm^2** .

- Calculá el core area necesario para una utilización del 60%.
- Calculá las dimensiones del core si querés un aspect ratio (altura/ancho) de 1.0 (cuadrado).
- Calculá el die area si usás un core offset de $\{3, 3, 3, 3\} \mu m$.
- Escribí el comando `initialize_floorplan` con esos valores.
- ¿Qué pasa si aumentás la utilización al 80%? ¿Y al 90%?

Respuestas:

- Core area = $3200/0.6 = 5333 \mu m^2$
- lado = $\sqrt{5333} \approx 73 \mu m$, entonces core = $73 \times 73 \mu m$
- Die = $(73 + 6) \times (73 + 6) = 79 \times 79 \mu m$
- `initialize_floorplan -core_size {73 73} -core_offset {3 3 3 3}`
- Al 80%: core = $4000 \mu m^2$ (63×63). Al 90%: $3556 \mu m^2$ (60×60) — alto riesgo de congestión.

9.2 Ejercicio 2: Análisis de placement

Después de correr `place_opt` en el filtro IIR, obtenés este QoR:

```
WNS = -0.082 ns (VIOLATED)
TNS = -0.214 ns
Area = 4250.3 um^2
InstCnt = 287
LVT percent = 94%
```

- ¿Qué significa WNS negativo? ¿El chip funciona?
- El sintetizador ya cerró timing — ¿por qué viola post-placement?
- 94% de celdas son LVT. ¿Qué intenta hacer el sintetizador?
- ¿Qué opciones tenés para cerrar timing? Nombrá al menos 3.
- Si bajás la frecuencia de 500MHz a 400MHz, ¿el problema desaparece? ¿Por qué?

Respuestas:

- a) $WNS = -0.082ns$ significa que el path más lento llega 82ps tarde. El chip funcionaría con errores.
- b) Post-síntesis los wires son ideales (sin delay). Post-placement se estiman los parásitos reales: los wires largos agregan delay que no estaba en síntesis.
- c) Está usando las celdas más rápidas disponibles para tratar de cerrar timing.
- d) (1) Bajar la frecuencia. (2) Hacer pipeline (agregar registros para cortar el critical path). (3) Cambiar la arquitectura del sumador. (4) Aumentar el floorplan (más espacio = wires más cortos).
- e) A 400MHz el período es 2.5ns. $2.5 - 2.0 = 0.5ns$ de margen extra. Como la violación es solo 82ps, sí — a 400MHz probablemente cierre.

9.3 Ejercicio 3: Clock Tree

Tenés un diseño con las siguientes características:

- Clock a 1 GHz (período = 1.0ns)
 - $T_{setup} = 0.10ns$
 - $T_{clk \rightarrow Q} = 0.12ns$
 - $T_{comb} = 0.60ns$
 - Skew del clock tree = 0.08ns (FF receptor llega el clock más tarde)
- a) Calculá el slack de setup **sin** considerar el skew.
 - b) Calculá el slack de setup **con** el skew. ¿Mejora o empeora?
 - c) ¿Qué valor máximo de skew podés tolerar antes de que setup viole?
 - d) Calculá el slack de hold con $T_{hold} = 0.04ns$ y el mismo skew de 0.08ns.
 - e) ¿El skew ayuda o perjudica el hold check?

Respuestas:

- a) $Slack = T_{period} - T_{clk \rightarrow Q} - T_{comb} - T_{setup} = 1.0 - 0.12 - 0.60 - 0.10 = +0.18ns$
- b) Con skew útil (receptor llega tarde): $Slack = 0.18 + 0.08 = +0.26ns$. Mejora porque el receptor tiene más tiempo.
- c) El skew puede ser hasta 0.18ns antes de que viole (slack = 0).
- d) $Hold\ slack = T_{clk \rightarrow Q} + T_{comb} - T_{hold} - skew = 0.12 + 0.60 - 0.04 - 0.08 = +0.60ns$. Positivo, no viola.
- e) El skew *perjudica* el hold: reduce el margen porque el dato llega antes de lo que el receptor espera.

9.4 Ejercicio 4: Routing y DRC

Mirando el reporte de routing del filtro FIR obtenés:

```
Total open nets: 3
DRC violations: 12
  - Spacing violations (M2): 7
  - Width violations (M3): 3
  - Via enclosure (VIA23): 2
Average horizontal track utilization: 78.3%
Peak    horizontal track utilization: 103.2%
```

- ¿Qué significa “3 open nets”? ¿El chip puede fabricarse así?
- La utilización peak de 103.2% es problemática. ¿Qué significa?
- ¿Cuál es la causa más probable de la congestión?
- Nombrá dos formas de reducir la congestión sin cambiar la lógica del diseño.
- Un error de “Via enclosure” significa que el wire no envuelve correctamente la via. ¿Cómo lo arregla FC automáticamente?

Respuestas:

- Hay 3 conexiones del netlist que el router no pudo rutear. El chip NO puede fabricarse — hay circuito abierto en 3 nets.
- Significa que la demanda de tracks en esa zona supera la capacidad — hay más wires para pasar que espacio físico disponible.
- Utilización alta (80%+) en el core deja poco espacio para los wires. O hay un macro o bloque que bloquea el routing en esa zona.
- (1) Bajar la utilización del core (floorplan más grande). (2) Redistribuir las celdas del área congestionada manualmente.
- FC tiene una regla de “via fixing” en `route_opt` que agrega metal extra para cumplir el enclosure mínimo.

9.5 Ejercicio 5: Comparación pre/post routing

Corré el flujo completo del filtro FIR (syn + pnr) y completá esta tabla:

Métrica	Post-Síntesis	Post-Placement	Post-Routing
WNS (ns)			
TNS (ns)			
Área (μm^2)			
% LVT			
# Celdas			
Potencia dinámica (mW)			

Preguntas a responder:

- a) ¿El WNS empeoró entre síntesis y routing? ¿Cuánto?
- b) ¿El área cambió entre síntesis y routing? ¿Por qué?
- c) ¿El % de LVT cambió? Si aumentó post-routing, ¿por qué?

9.6 Ejercicio Integrador: Filtro FIR de 19 taps — Flujo completo

Corré el flujo completo (syn + pnr) del filtro FIR que diseñaron en el TP.

Paso 1: Síntesis

```
make syn
```

Del reporte post-síntesis respondé:

- a) ¿Cuántas celdas tiene el diseño sintetizado?
- b) ¿Cuántos flip-flops? (en el reporte de área, buscá DFFX)
- c) ¿Cuál es el slack? ¿Qué frecuencia máxima soporta?
- d) ¿Qué porcentaje de celdas son LVT vs HVT?

Paso 2: Floorplan

Con el área de síntesis, calculá el tamaño del core para 60% de utilización y editá el floorplan.tcl.

Paso 3: P&R

```
make pnr
```

Del reporte post-P&R respondé:

- a) ¿El timing cerró? Compará WNS pre y post routing.
- b) ¿Cuántos buffers insertó el CTS?
- c) ¿Cuál es la longitud total de wire? ¿En qué capas están los wires más largos?
- d) ¿Hay DRC violations? Si hay, de qué tipo.
- e) ¿Cuál es la potencia total? ¿Qué porcentaje es dinámica vs leakage?

Paso 4: Análisis en la GUI

Abrí FC con `make gui-syn` y explorá el layout del filtro:

- a) Identificá los registros del shift register (los 18 DFFs de delay).
- b) Identificá la cadena de multiplicadores/sumadores.
- c) Usá **Highlight** → **Clock Tree** para ver el árbol de clock.
- d) Deshabilitá M3 y M4 en el panel de capas. ¿Qué ven?

10 Resumen del flujo P&R

Paso	Comando	Qué hace
Floorplan	<code>initialize_floorplan</code>	Define el área del chip, utilización, offset
Pin placement	<code>place_pins</code>	Ubica los pines de I/O en los bordes
Power	<code>connect_pg_net</code>	Conecta las redes de VDD/VSS
Placement	<code>place_opt</code>	Ubica las celdas y optimiza timing
CTS	<code>clock_opt</code>	Construye el árbol de clock
Routing	<code>route_auto</code>	Rutea todos los wires
Opt post-route	<code>route_opt</code>	Optimiza timing y DRC post-routing
Reports	<code>report_*</code>	Timing, área, potencia, QoR
Outputs	<code>write_gds</code>	GDS para fabricación

Checklist de P&R

Antes de decir que el P&R está completo, verificar:

1. $WNS \geq 0$ en todos los escenarios (setup y hold)
2. DRC violations = 0
3. Open nets = 0
4. LVS clean (Layout vs Schematic)
5. IR drop dentro del límite ($< 10\%$ de VDD)
6. Congestion peak $< 90\%$